



آزمایشگاه معماری کامپیوتر

نیم سال سوم ۱۴۰۴-۰۵

استاد: دکتر حمید سربازی آزاد

دانشکده مهندسی کامپیوتر

معین آعلی - ثمین اکبری

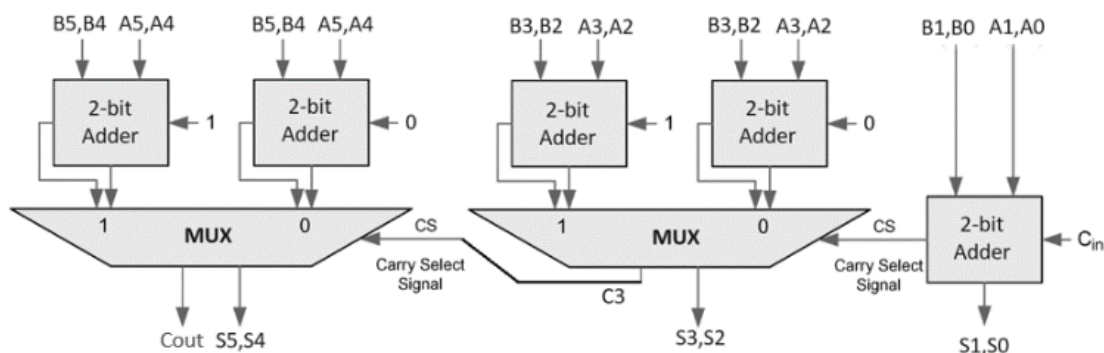
جمع کننده انتخاب رقم نقالی

آزمایش دوم

خلاصه

در این آزمایش یک جمع کننده ۶ بیتی با تکنیک Carry Select در نرم افزار Logisim طراحی و پیاده سازی شده است. مدار به صورت سلسله مراتبی و از پایین به بالا ساخته شده: یک تمام جمع کننده ۱ بیتی (full_adder)، که پایه ی ساخت یک جمع کننده ۲ بیتی ripple-carry (adder2) است، و در نهایت مدار اصلی (main) با ترکیب پنج نمونه از این جمع کننده ۲ بیتی به همراه چهار قطعه ی آماده ی Multiplexer از کتابخانه ی Logisim ساخته شده است. این مدار Hybrid از هر دو ابزار جمع کننده ی معمولی و مالتی پلکسر برای بهبود سرعت جمع نسبت به یک جمع کننده ی ripple-carry صرف استفاده می کند.

دیاگرام جمع کننده انتخاب رقم نقالی



هدف آزمایش

هدف از این آزمایش، آشنایی عملی با طراحی مدارهای Hybrid (ترکیبی از جمع کننده و مدار انتخاب) به منظور بهبود سرعت عمل جمع است. برخلاف جمع باینری معمولی به روش ripple-carry که در آن هر طبقه باید منتظر رقم نقالی طبقه ی قبل بماند، در تکنیک Carry Select با محاسبه ی موازی و انتخاب بعدی، این انتظار تا حد زیادی حذف می شود. اهداف مشخص این آزمایش عبارت اند از:

- پیاده سازی یک تمام جمع کننده ۱ بیتی (Full Adder) با گیت های پایه.
- ساخت یک جمع کننده ۲ بیتی ripple-carry از دو نمونه تمام جمع کننده.
- طراحی مدار جمع ۶ بیتی با تکنیک Carry Select، با استفاده از جمع کننده های ۲ بیتی و قطعه ی آماده ی Multiplexer.
- بررسی و درک مبادله ی سرعت در برابر سخت افزار در تکنیک Carry Select.

چرا Carry Select؟

در یک جمع‌کننده ripple-carry معمولی، هر طبقه باید منتظر رقم نقلی طبقه‌ی قبل بماند تا جمع خودش را شروع کند؛ تأخیر کل با تعداد بیت‌ها به صورت خطی رشد می‌کند. در Carry Select، هر گروه از بیت‌ها را دوبار و به موازات جمع می‌زنیم - یک بار با فرض $C_{in} = 0$ و یک بار با فرض $C_{in} = 1$ - و وقتی رقم نقلی واقعی از طبقه‌ی قبل رسید، فقط با یک Multiplexer جواب درست را انتخاب می‌کنیم. این کار محاسبه‌ی هر گروه را از انتظار برای رقم نقلی گروه قبلی آزاد می‌کند و تأخیر کلی مدار را کم می‌کند (به قیمت سخت‌افزار بیشتر، چون هر گروه به جز اولی باید دوبار جمع زده شود).

تمام جمع‌کننده‌ی ۱ بیتی

پایه	جهت	پهنا
C_{in}, B, A	ورودی	۱ بیت
C_{out}, Sum	خروجی	۱ بیت

مدار با ۲ گیت XOR، ۲ گیت AND و ۱ گیت OR ساخته شده است:

$$X_1 = A \oplus B$$

نیمه جمع، بدون در نظر گرفتن رقم نقلی ورودی.

$$P_1 = A \wedge B$$

آیا از جمع A و B به تنهایی رقم نقلی تولید می‌شود؟

$$Sum = X_1 \oplus C_{in}$$

جمع نهایی سه ورودی.

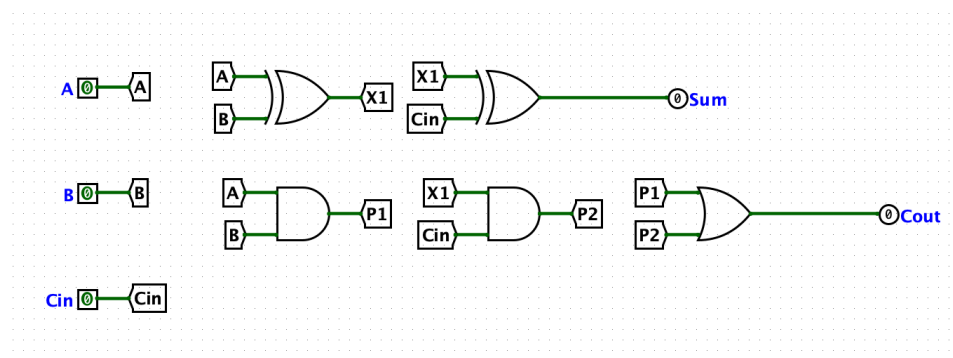
$$P_2 = X_1 \wedge C_{in}$$

آیا از جمع X_1 و C_{in} رقم نقلی تولید می‌شود؟

$$C_{out} = P_1 \vee P_2$$

رقم نقلی خروجی نهایی.

A	B	C_{in}	Sum	C_{out}
۰	۰	۰	۰	۰
۰	۰	۱	۱	۰
۰	۱	۰	۱	۰
۰	۱	۱	۰	۱
۱	۰	۰	۱	۰
۱	۰	۱	۰	۱
۱	۱	۰	۰	۱
۱	۱	۱	۱	۱



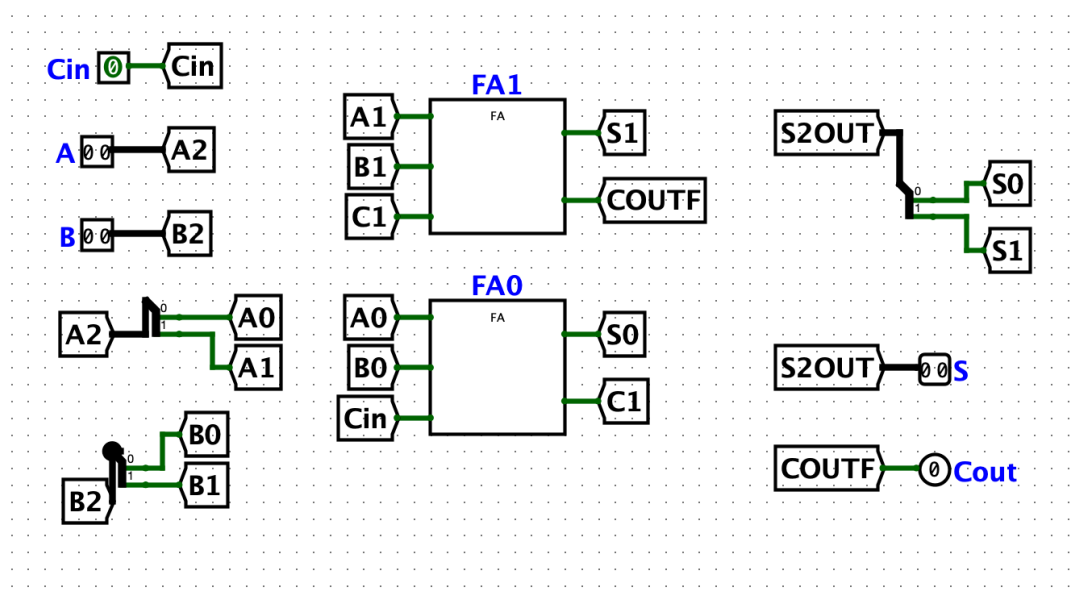
جمع‌کننده ۲ بیتی

پایه	جهت	پهنا
B, A	ورودی	۲ بیت
Cin	ورودی	۱ بیت
S	خروجی	۲ بیت
Cout	خروجی	۱ بیت

ورودی‌های A و B با یک تفکیک‌کننده به بیت‌های تکی A_0, A_1 و B_0, B_1 شکسته می‌شوند و در زنجیره‌ای از ۲ تمام‌جمع‌کننده (FA0, FA1) به صورت ripple-carry جمع زده می‌شوند:

نمونه	بیت	ورودی‌ها	خروجی جمع	رقم نقلی
FA0	۰ (کم‌ارزش)	A_0, B_0, C_{in}	S_0	C_1
FA1	۱ (پرازش)	A_1, B_1, C_1	S_1	Cout

بیت‌های S_1, S_0 با یک تفکیک‌کننده ترکیبی دوباره به باس ۲ بیتی پایه‌ی خروجی S وصل می‌شوند و رقم نقلی خروجی FA1 مستقیماً به پایه‌ی Cout این مدار وصل است. این جمع‌کننده ۲ بیتی، بلوک سازنده‌ی اصلی مدار Carry Select است و در مدار اصلی پنج بار نمونه‌سازی می‌شود.



مدار اصلی

پایه	جهت	پهنا
B, A	ورودی	۶ بیت
Cin	ورودی	۱ بیت
S	خروجی	۶ بیت
Cout	خروجی	۱ بیت

مدار به ۳ گروه ۲ بیتی تقسیم می‌شود. ابتدا A و B با یک تفکیک‌کننده ۶ به ۶ بیت‌های تکی شکسته می‌شوند و بعد هر دو بیت هم‌گروه دوباره با یک تفکیک‌کننده ترکیبی به یک باس ۲ بیتی تبدیل می‌شوند تا وارد جمع‌کننده ۲ بیتی شوند:

گروه	بیت‌ها	باس A	باس B
گروه ۰	۰، ۱ (کم‌ارزش)	GA0	GB0
گروه ۱	۲، ۳	GA1	GB1
گروه ۲	۴، ۵ (پرارزش)	GA2	GB2

همچنین دو ثابت ساخته شده است: ZERO1 (ثابت ۰، برای فرض $C_{in} = 0$) و ONE1 (ثابت ۱، برای فرض $C_{in} = 1$).

گروه ۰ (بیت‌های ۰، ۱) - بدون نیاز به انتخاب. چون این کم‌ارزش‌ترین گروه است، فقط یک رقم نقلی ورودی دارد (رقم نقلی واقعی Cin مدار) و نیازی به Multiplexer نیست:

نمونه	A	B	Cin	خروجی
AD0	GA0	GB0	Cin (واقعی)	$S_1 S_0$ ، رقم نقلی C_1

رقم نقلی خروجی این طبقه، C_1 ، به عنوان سیگنال انتخاب گروه بعدی استفاده می‌شود.

گروه ۱ (بیت‌های ۲، ۳) - انتخاب بر اساس C_1 . جمع این گروه دوبار و به موازات با هر دو فرض ممکن برای رقم نقلی ورودی حساب می‌شود:

نمونه	A	B	Cin	خروجی
AD1_0 (فرض بدون نقلی)	GA1	GB1	ZERO1	$SG1_0 = S_3 S_2$ ، رقم نقلی $C3_0$
AD1_1 (فرض با نقلی)	GA1	GB1	ONE1	$SG1_1 = S_3 S_2$ ، رقم نقلی $C3_1$

حالا با رسیدن C_1 واقعی، دو قطعه‌ی آماده‌ی Multiplexer از کتابخانه‌ی Logisim بین این دو جواب یکی را انتخاب می‌کنند:

مالتی‌پلکسر	D0 ($C_{in} = 0$)	D1 ($C_{in} = 1$)	Sel	خروجی Y
MUXS1 (۲ بیت)	SG1_0	SG1_1	C_1	S_3, S_2 نهایی
MUXC1 (۱ بیت)	C3_0	C3_1	C_1	C_3

سیگنال C_3 (خروجی MUXC1) هم رقم نقلی این گروه است، هم سیگنال انتخاب گروه بعدی.

گروه ۲ (بیت‌های ۴، ۵) - انتخاب بر اساس C_3 . دقیقاً همان الگوی گروه ۱، فقط با C_3 به جای C_1 :

نمونه	A	B	Cin	خروجی
AD2_0	GA2	GB2	ZERO1	SG2_0 = S_5S_4 , رقم نقلی C5_0
AD2_1	GA2	GB2	ONE1	SG2_1 = S_5S_4 , رقم نقلی C5_1

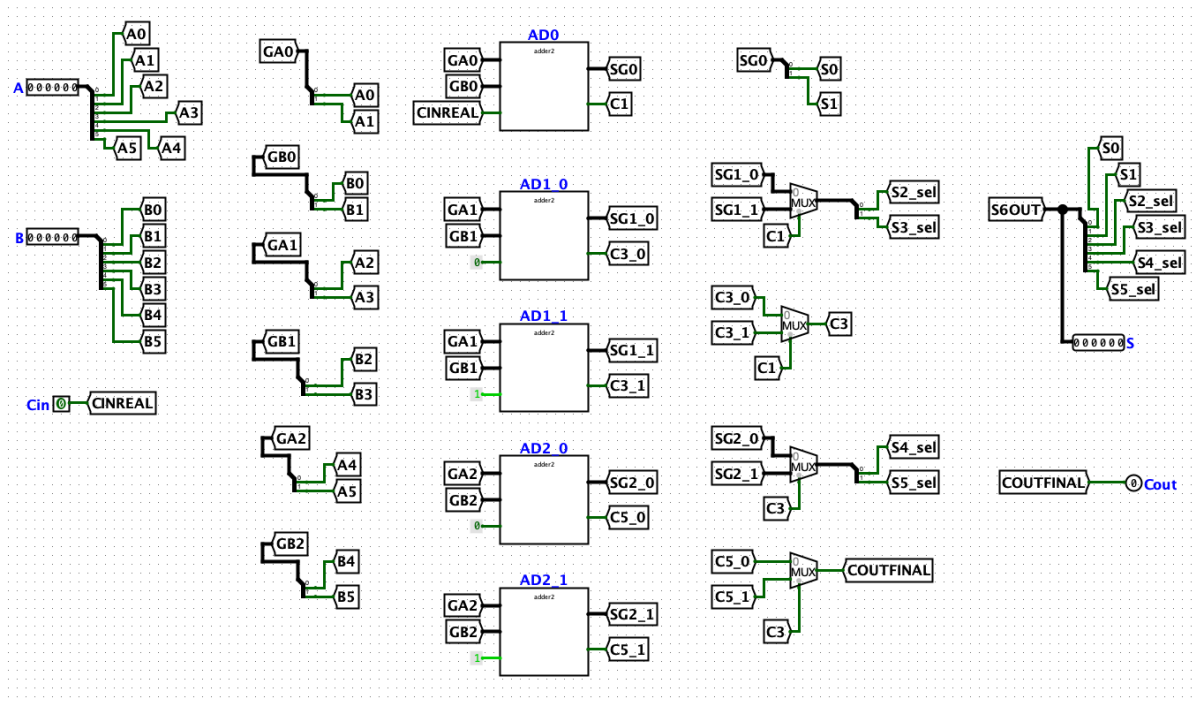
مالتی پلکسر	D0 ($C_{in} = 0$)	D1 ($C_{in} = 1$)	Sel	خروجی Y
MUXS2 (۲ بیت)	SG2_0	SG2_1	C_3	S_5, S_4 نهایی
MUXC2 (۱ بیت)	C5_0	C5_1	C_3	COUTFINAL

سیگنال COUTFINAL (خروجی MUXC2) رقم نقلی نهایی کل مدار است و مستقیماً به پایه‌ی خروجی Cout وصل می‌شود.

مونتاز نهایی. شش بیت $S_0, S_1, S_2, S_3, S_4, S_5$ با یک تفکیک‌کننده‌ی ۶ تایی دوباره به باس خروجی S ترکیب می‌شوند.

چرا این معماری سریع‌تر است؟ چون $AD1_0, AD1_1, AD2_0, AD2_1$ هیچ‌کدام منتظر رقم نقلی واقعی نمی‌مانند (هرکدام رقم نقلی ثابت خودشان را دارند)، همه‌ی این ۴ جمع‌کننده می‌توانند کاملاً هم‌زمان و موازی با $AD0$ شروع به محاسبه کنند. تنها چیزی که در انتها زمان می‌برد، عبور C_1 و سپس C_3 از میان یک لایه Multiplexer است – که خیلی سریع‌تر از عبور رقم نقلی واقعی از میان ۶ تمام‌جمع‌کننده‌ی پشت‌سرهم (مثل حالت ripple-carry ساده) است.

مرحله	چه چیزی حساب می‌شود	منتظر چه سیگنالی است
۱	$AD0$ جمع بیت‌های ۰، ۱ را با C_{in} واقعی حساب می‌کند و C_1 را تولید می‌کند	فقط C_{in} واقعی مدار
۲	$AD1_0$ و $AD1_1$ جمع بیت‌های ۲، ۳ را برای هر دو حالت ممکن رقم نقلی از پیش حساب می‌کنند	هیچ‌چیز – هم‌زمان با $AD0$
۳	$MUXC1/MUXS1$ با C_1 بین دو جواب مرحله‌ی ۲ انتخاب می‌کنند و C_3 را می‌سازند	فقط رسیدن C_1
۴	$AD2_0$ و $AD2_1$ جمع بیت‌های ۴، ۵ را برای هر دو حالت از پیش حساب می‌کنند	هیچ‌چیز
۵	$MUXC2/MUXS2$ با C_3 بین دو جواب مرحله‌ی ۴ انتخاب می‌کنند و Cout نهایی را می‌سازند	فقط رسیدن C_3



نتایج تست

هر سه مدار با بردار تست و از طریق خط فرمان Logisim در حالت headless تست شدند:

```
CE103-CAL/Assignments/Lab02-Carry-Select-Adder on ʘ main [!?]
● > bash run_tests.sh
circ : carry_select_adder.circ
java : /opt/homebrew/opt/openjdk@21/bin/java
jar : /Applications/Logisim-evolution.app/Contents/app/logisim-evolution-4.1.0-all.jar
vecs : 3

=== tests_adder2.vec → circuit 'adder2' ===
Loading test vector "tests_adder2.vec"...
Running 32 vectors...
32
Passed: 32, Failed: 0
PASS: tests_adder2.vec

=== tests_full_adder.vec → circuit 'full_adder' ===
Loading test vector "tests_full_adder.vec"...
Running 8 vectors...
8
Passed: 8, Failed: 0
PASS: tests_full_adder.vec

=== tests_main_exhaustive.vec → circuit 'main' ===
Loading test vector "tests_main_exhaustive.vec"...
Running 8192 vectors...
8192
Passed: 8192, Failed: 0
PASS: tests_main_exhaustive.vec

summary: 3/3 passed, 0 failed
```

بردارهای تست داخل فایل‌های *.vec قرار دارند و با اسکریپت run_tests.sh به صورت خودکار روی هر سه مدار اجرا می‌شوند.